

Mock Research Document for PDF Extraction Testing

Omkar Patil¹, Alex Chen²

¹Independent Researcher ²Data Systems Lab

Abstract — This mock document contains common formatting structures encountered in research papers and technical documents. It includes headings, lists, tables, pseudo two-column layouts, footnotes, code blocks, captions, and mixed text styles. The goal is to evaluate how well document parsers and OCR pipelines preserve logical reading order and structure for downstream RAG pipelines.

1. Introduction

Modern retrieval-augmented generation (RAG) systems depend heavily on reliable document parsing. However, PDFs often contain complex layouts such as multi-column text, tables, mathematical expressions, and captions. Incorrect ordering during extraction can lead to corrupted semantic chunks.

2. Background

PDFs are primarily designed for visual fidelity rather than semantic structure. Libraries often rely on positional heuristics to infer reading order.

Challenges include:

- Multi-column layouts
- Figures with captions
- Tables with merged cells
- Footnotes and references

3. Typical Parsing Pipeline

A common pipeline for RAG ingestion consists of the following stages:

- PDF text extraction
- Layout reconstruction
- Semantic chunking
- Embedding generation
- Vector database indexing

Table 1: Example Structured Data

Model	Parameters	Year	Notes
GPT-3	175B	2020	Large language model
BERT	340M	2018	Bidirectional transformer
ResNet-50	25M	2015	Computer vision CNN

Figure 1: Example diagram placeholder. In real documents, figures may appear between paragraphs and disrupt reading order during naive extraction.

Code Example

```
def chunk_document(text, chunk_size=500):  
    words = text.split()
```

```
chunks = []
for i in range(0, len(words), chunk_size):
    chunk = " ".join(words[i:i+chunk_size])
    chunks.append(chunk)
return chunks
```

¹Footnote example: Footnotes are often extracted in incorrect order depending on how the parser handles bottom-of-page elements.